

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

ENGENHARIA DE SOFTWARE

JOVI STUDY LENS

BRUCE LI YAN TING RM: 569168

FELIPE RORATO COSTA RM: 570419

GIOVANNA ANDRADE PAREJAS RM: 569041

LAURA CARDIN MIRILLI RM: 570841

TURMA: 1ESPI

SPRINT 4 - COMPUTATIONAL THINKING WITH PYTHON

SÃO PAULO

2026

BRUCE LI YAN TING
FELIPE RORATO COSTA
GIOVANNA ANDRADE PAREJAS
LAURA CARDIN MIRILLI

SPRINT 4 - COMPUTATIONAL THINKING WITH PYTHON

Trabalho acadêmico apresentado ao Centro Universitário FIAP como parte dos requisitos para aprovação na disciplina de Computational Thinking With Python do curso de Engenharia de Software.

Orientador: Patrícia Maura Angelini

SÃO PAULO
2026

SUMÁRIO

1. DESCRIÇÃO DO PROJETO.....4
1.1 Objetivo geral.....4
1.2 Justificativa.....4
2. FUNCIONALIDADES IMPLEMENTADAS.....6
2.1 Digitalizar Foto de Aula.....6
2.2 Copiar conteúdo.....6
2.3 Traduzir conteúdo.....6
2.4 Biblioteca de materiais.....7
3. JUSTIFICATIVA DE ALTERAÇÕES REALIZADAS EM RELAÇÃO À SPRINT ANTERIOR.....7
3.1 Removido.....8
3.2 Modificado.....8
3.3 Adicionado.....8

1. DESCRIÇÃO DO PROJETO

O JOVI StudyLens é um protótipo em Python desenvolvido como parte do Challenge FIAP 2026, simulando uma funcionalidade do smartphone Jovi voltada ao estudante em tempo integral. O sistema transforma registros fotográficos feitos em sala de aula (lousas, slides, cadernos) em material de estudo organizado, pesquisável e reutilizável, acessível diretamente pelo terminal. Nesta Sprint 4, o projeto evoluiu com a implementação de persistência de dados em arquivos JSON: os materiais digitalizados e o histórico de traduções passam a ser salvos automaticamente em disco (materiais.json e historico_traducoes.json) e recarregados a cada nova execução do programa. A funcionalidade de tradução passou a consumir diretamente uma API REST externa (MyMemory), por meio de requisições HTTP feitas com a biblioteca requests. A estrutura interna dos materiais também foi reformulada, migrando de listas posicionais para dicionários nomeados, e foram adicionadas anotações de tipo (type hints) e tratamento de exceções (try/except) nos pontos sujeitos a falhas externas, sem alteração das regras de negócio já implementadas nas sprints anteriores.

1.1 Objetivo geral

Eliminar a dor do estudante que acumula fotos de aula na galeria do celular sem organização ou utilidade prática, oferecendo um fluxo estruturado de captura, armazenamento, tradução e acesso ao conteúdo. Nesta sprint, esse fluxo passa a ser persistente: o que o estudante digitaliza e traduz deixa de existir apenas durante a execução do programa e passa a ser salvo em disco, disponível na próxima vez que ele abrir o JOVI StudyLens.

1.2 Justificativa

Estudantes frequentemente fotografam conteúdos em sala, mas essas imagens se perdem entre outras fotos no dispositivo, sem categorização por matéria, tipo ou data. O JOVI StudyLens resolve esse problema centralizando o conteúdo em uma biblioteca pessoal, com funcionalidades que vão além do simples armazenamento: o aluno pode copiar o texto extraído, traduzi-lo para outros idiomas e consultá-lo a qualquer momento de forma organizada. Na Sprint 4, o foco foi tornar essa biblioteca confiável entre usos: até então, fechar o programa significava perder tudo o que havia sido digitalizado ou traduzido na

sessão, o que contrariava a própria proposta de centralizar o material de estudo do aluno. Com a persistência em arquivos JSON, os dados passam a sobreviver ao encerramento do programa, e o tratamento de exceções garante que o sistema continue funcionando de forma previsível mesmo diante de um arquivo ausente, corrompido, ou de uma falha temporária no serviço de tradução, cenários que antes não eram tratados e podiam interromper o uso do protótipo.

2. FUNCIONALIDADES IMPLEMENTADAS

2.1 Digitalizar Foto de Aula

Simula o processo de captura e extração de texto de um material fotográfico. O usuário informa o tipo de material (lousa, slide, caderno), define um título, seleciona a matéria correspondente e insere o conteúdo textual extraído, que em uma versão com IA seria gerado automaticamente via OCR. O material é então salvo na biblioteca com um ID único, permitindo recuperação posterior. Nesta sprint, esse salvamento passou a ser persistente: o material gravado é imediatamente escrito no arquivo materiais.json, ficando disponível mesmo após o encerramento e reabertura do programa.

Alinhamento ao objetivo: reproduz a etapa de digitalização do fluxo Fotografia → IA processa → Aluno aprende, o núcleo da proposta do produto.

2.2 Copiar conteúdo

Permite ao usuário selecionar qualquer material salvo na biblioteca e copiar seu conteúdo textual diretamente para a área de transferência do sistema, utilizando a biblioteca pyperclip. O texto copiado pode ser colado em qualquer outro aplicativo, como editor de texto, e-mail, etc. Caso a área de transferência não esteja disponível no sistema operacional, o programa exibe uma mensagem de erro e retorna ao menu, sem ser interrompido.

Alinhamento ao objetivo: elimina a necessidade de digitar repetidas vezes o conteúdo capturado, tornando o material imediatamente reutilizável.

2.3 Traduzir Conteúdo

Oferece tradução de textos do Português para o Inglês por meio do consumo direto da API REST do MyMemory, utilizando a biblioteca requests. O programa envia uma requisição HTTP GET com o texto e o par de idiomas, recebe a resposta em formato JSON e extrai dela o texto traduzido. O usuário pode traduzir um texto digitado manualmente ou selecionar diretamente um material da biblioteca, respeitando o limite de 500 caracteres por requisição imposto pela versão gratuita da API. O resultado é exibido lado a lado com o original, registrado no histórico de traduções (persistido em historico_traducoes.json) e, opcionalmente, salvo como novo material na biblioteca no idioma de destino. Nesse caso, o novo material herda a matéria do material de origem, ou é classificado como “Outro” quando a tradução parte de um texto avulso. Falhas na comunicação com a API (sem conexão, tempo

esgotado, limite de uso excedido ou resposta inválida) são tratadas com uma mensagem clara ao usuário, em vez de interromper o programa.

Alinhamento ao objetivo: amplia o alcance do material de estudo, permitindo que o aluno utilize o conteúdo capturado em contextos bilíngues, tal como preparação para provas, leitura de artigos ou estudo de idiomas.

2.4 Biblioteca de materiais

Central de acesso a todos os materiais digitalizados, incluindo três exemplos pré-carregados (História, Filosofia e Matemática) que agora são carregados a partir do arquivo materiais.json, e não mais recriados do zero a cada execução. Oferece cinco modos de navegação: visualização completa do acervo, filtragem por matéria, busca por palavra-chave no título ou conteúdo, leitura integral de um material selecionado e resumo feito por IA para os materiais de exemplo. Nas opções de visualização completa, filtragem e busca, o sistema também oferece acesso rápido ao conteúdo: após exibir a lista, pergunta se o estudante deseja abrir algum material específico digitando o ID correspondente, restringindo a busca apenas aos itens daquela lista exibida.

Alinhamento ao objetivo: resolve diretamente a dor central do projeto, uma vez que o conteúdo fotografado deixa de estar perdido na galeria e passa a ser acessível de forma organizada, categorizada e pesquisável — agora de forma duradoura entre uma sessão de uso e outra.

3. JUSTIFICATIVA DE ALTERAÇÕES REALIZADAS EM RELAÇÃO À SPRINT ANTERIOR

Nesta sprint, o projeto evoluiu em três frentes: a implementação de persistência de dados em arquivos JSON, o consumo direto de uma API REST externa para tradução e o reforço da robustez do sistema por meio de tratamento de exceções e tipagem estática. Nenhuma das quatro funcionalidades centrais (Digitalizar, Copiar, Traduzir e Biblioteca) foi removida ou teve sua regra de negócio original alterada.

3.1 Removido

- Dependência da biblioteca deep-translator, substituída pelo consumo direto da API do MyMemory via requests.
- Lista de materiais escrita diretamente no código (hardcoded), substituída pelo carregamento a partir do arquivo materiais.json.
- Variável global “saida”, que não era utilizada em nenhum ponto do código.

3.2 Modificado

- Estrutura de dados dos materiais: materiais deixou de ser uma lista de listas posicional ([id, titulo, materia, tipo, conteudo, idioma], acessada por índice como material[1]) e passou a ser uma lista de dicionários ({"id": ..., "titulo": ..., ...}, acessada por chave como material["titulo"]). A mudança afetou todas as funções que leem ou escrevem em materiais.
- Tradução: a chamada à biblioteca GoogleTranslator foi substituída por uma requisição HTTP GET direta ao endpoint https://api.mymemory.translated.net/get, com leitura e validação da resposta JSON. A troca de serviço também resolveu o erro de limite de requisições (TooManyRequests) que a API não oficial do Google Translate passou a apresentar.
- Matéria do material salvo após tradução: antes fixa como "Tradução" (categoria inexistente em MATERIAS_DISPONIVEIS), agora herda a matéria do material de origem ou usa "Outro" para textos avulsos.
- Contador de próximo ID (proximo_id): antes fixo em 4, agora calculado a partir do maior ID já salvo (max(id) + 1), evitando IDs repetidos entre execuções.
- Validação de números em input_validado() e perguntar_acesso_material(): a verificação manual com .isdigit() foi substituída por conversão com int() dentro de try/except ValueError.
- Menu principal: o limite máximo aceito foi corrigido de max_val=5 para max_val=4, e a palavra “sair” deixou de ser aceita como opção nessa tela (permite_sair=False), já que o menu possui sua própria opção [0] Sair.
- Leitura de conteúdo na Biblioteca (opção 4): passou a reutilizar a função exibir_conteudo_material(), eliminando código duplicado.

- Filtro por matéria: as matérias passaram a ser exibidas em ordem alfabética (sorted), mantendo a mesma numeração entre execuções.
- Mensagem de boas-vindas: passou a exibir a quantidade real de materiais na biblioteca, em vez do texto fixo “3 materiais de exemplo”.

3.3 Adicionado

- Persistência de dados em JSON: funções carregar_materiais(), salvar_materiais(), carregar_historico() e salvar_historico(). Os dados são salvos automaticamente a cada alteração, sem necessidade de encerrar o programa.
- Caminhos dos arquivos JSON definidos com pathlib.Path em relação à pasta do app.py, garantindo que os dados sejam lidos e gravados no mesmo local independentemente do diretório de onde o programa é executado.
- Tratamento de exceções (try/except):
    - no carregamento dos arquivos JSON (FileNotFoundError, json.JSONDecodeError e ValueError para estrutura inválida), com recriação automática a partir dos dados padrão;
    - na requisição à API de tradução (requests.exceptions.RequestException, ValueError e KeyError), cobrindo falha de conexão, tempo esgotado, erro HTTP e resposta fora do formato esperado;
    - na cópia para a área de transferência (pyperclip.PyperclipException).
- Validação do tamanho do texto antes da tradução (limite de 500 caracteres da API gratuita).
- Verificação de biblioteca vazia na funcionalidade Copiar e de material inexistente no Resumo.
- Uso de rich.markup.escape() na exibição de textos digitados pelo usuário, evitando que caracteres como [e] sejam interpretados como formatação pela biblioteca Rich.
- Type hints em parâmetros e retornos de todas as funções, incluindo o alias de tipo Material = dict[str, str | int] e anotações nas variáveis globais.
- Dependência requests, adicionada ao requirements.txt no lugar de deep-translator.